

Laporan Testing Quality Assurance

Informasi Pengujian

Item	Detail
Project	Task Management API
Teknologi	Golang, JWT Authentication, Redis Cache
Base URL	http://localhost:8080
Tools Testing	Postman
Status Testing	Berhasil dengan catatan minor validation issue

Hasil Pengujian API

1. Health Check

Endpoint

GET /health

Hasil

Server berhasil merespons dengan status normal.

 http://localhost:8080/health

Save

Share

GET

http://localhost:8080/health

Send

Docs

Params

Authorization

Headers (11)

Body

Scripts

Tests

Settings

Cookies

Query Params

	Key	Value	Description		Bulk Edit
	Key	Value	Description		

Body

Cookies

Headers (3)

Test Results

200 OK

62 ms

178 B

{}

JSON

Preview

Visualize

1

2

3

4

{

"status": "ok",

"timestamp": "2026-05-07T23:26:56+07:00"

}

Response

```
{  
  "status": "ok",  
  "timestamp": "2026-05-07T23:26:56+07:00"  
}
```

Status

- HTTP Status: 200 OK
 - Result: SUCCESS
-

2. Registrasi User

Endpoint

POST /auth/register

Request Body

```
{  
  "username": "jab_pelindo",  
  "email": "jabyon@pelindo.co.id",  
  "password": "password123"  
}
```

Hasil

User berhasil didaftarkan ke sistem dan token JWT berhasil dibuat.

HTTP <http://localhost:8080/auth/register> Save Share

POST <http://localhost:8080/auth/register> Send

Docs Params Authorization Headers (11) Body Scripts Tests Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "username": "jab_pelindo",
3   "email": "jabyoan@pelindo.co.id",
4   "password": "password123"
5 }
```

Body Cookies Headers (3) Test Results 201 Created • 149 ms • 562 B

{ } JSON Preview Visualize

```
1 {
2   "data": {
3     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFnZCI6ImphYnVhYyYw5AcGVsaW5kby5jby5pZCI6ImV4cCI6MTc3ODI1ODQzOCwiaWF0IjoxNzc4MTcyMDM4LCJzZWUiOiJhbnRmZWQ1My0wNjRmLTRjYzQtOGE5Yy02YTJjMWRlMGY1ZmQlLCJ1c2VybmFtZSI6ImphYyY19wZWxpbmRvIn0.fGQ1stQNWuFltQFrgACFEUrz644tk3BYORAKtfu0gtg",
4     "user": {
5       "id": "a64fed53-064f-4cc4-8a9c-6a2c1de0f5fd",
6       "username": "jab_pelindo",
7       "email": "jabyoan@pelindo.co.id"
8     }
9   },
10  "message": "Akun berhasil didaftarkan"
11 }
```

Response

```
{
  "message": "Akun berhasil didaftarkan"
}
```

Status

- HTTP Status: 201 Created
- Result: SUCCESS

3. Login User

Endpoint

POST /auth/login

Request Body

```
{
  "email": "jabyon@pelindo.co.id",
  "password": "password123"
}
```

User berhasil login dan sistem mengembalikan JWT token untuk autentikasi endpoint task.

The screenshot shows a Postman interface with a POST request to `http://localhost:8080/auth/login`. The body is raw JSON:

```
{
  "email": "jabyoan@pelindo.co.id",
  "password": "password123"
}
```

The response status is 200 OK. The response body is JSON:

```
{
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWFnbnVYVW5AcGVsaW5kb3Yyby5pZCIsImV4cCI6MTc3ODI1ODUxMCwiaWF0IjoxNzc4MTEyMTEwLCJzdWIiOiJhbnRmZWQ1My0wNjRmLTRjYzQtOGESYy02YTJjMWRlMGY1ZmQiLCJpc2VybmFtZSI6ImphY19wZWxpbmRvIn0.4ctqGsT_dsnSFALcFe5VxaEy9QhZtuQJVw21lRlwe-4",
    "user": {
      "id": "a64fed53-064f-4cc4-8a9c-6a2c1de0f5fd",
      "username": "jab_pelindo",
      "email": "jabyoan@pelindo.co.id"
    }
  },
  "message": "Login berhasil"
}
```

Response

```
{
  "message": "Login berhasil"
}
```

Status

- HTTP Status: 200 OK
- Result: SUCCESS

4. Create Task

Endpoint

POST /tasks

Header

Authorization: Bearer <TOKEN>

Request Body

```
{
  "title": "Meeting Persiapan Deployment",
  "description": "Membahas infrastruktur server Pelindo",
  "status": "pending",
  "due_date": "2026-05-10"
}
```

Hasil

Task berhasil dibuat dan tersimpan ke database.

The screenshot displays a REST client interface with the following details:

- URL:** `http://localhost:8080/tasks`
- Method:** `POST`
- Request Body (raw):**

```
{
  "title": "Meeting Persiapan Deployment",
  "description": "Membahas infrastruktur server Pelindo",
  "status": "pending",
  "due_date": "2026-05-10"
}
```
- Response Status:** `201 Created` (14 ms, 439 B)
- Response Body (JSON):**

```
{
  "message": "Task created successfully",
  "task": {
    "id": "1d3ff770-6626-46c7-ae38-55a0a15f6b41",
    "title": "Meeting Persiapan Deployment",
    "description": "Membahas infrastruktur server Pelindo",
    "status": "pending",
    "due_date": "2026-05-10",
    "created_at": "2026-05-07T23:45:25+07:00",
    "updated_at": "2026-05-07T23:45:25+07:00"
  }
}
```

Response

```
{
  "message": "Task created successfully"
}
```

Status

- HTTP Status: 201 Created
 - Result: SUCCESS
-

5. Get Task List

Endpoint

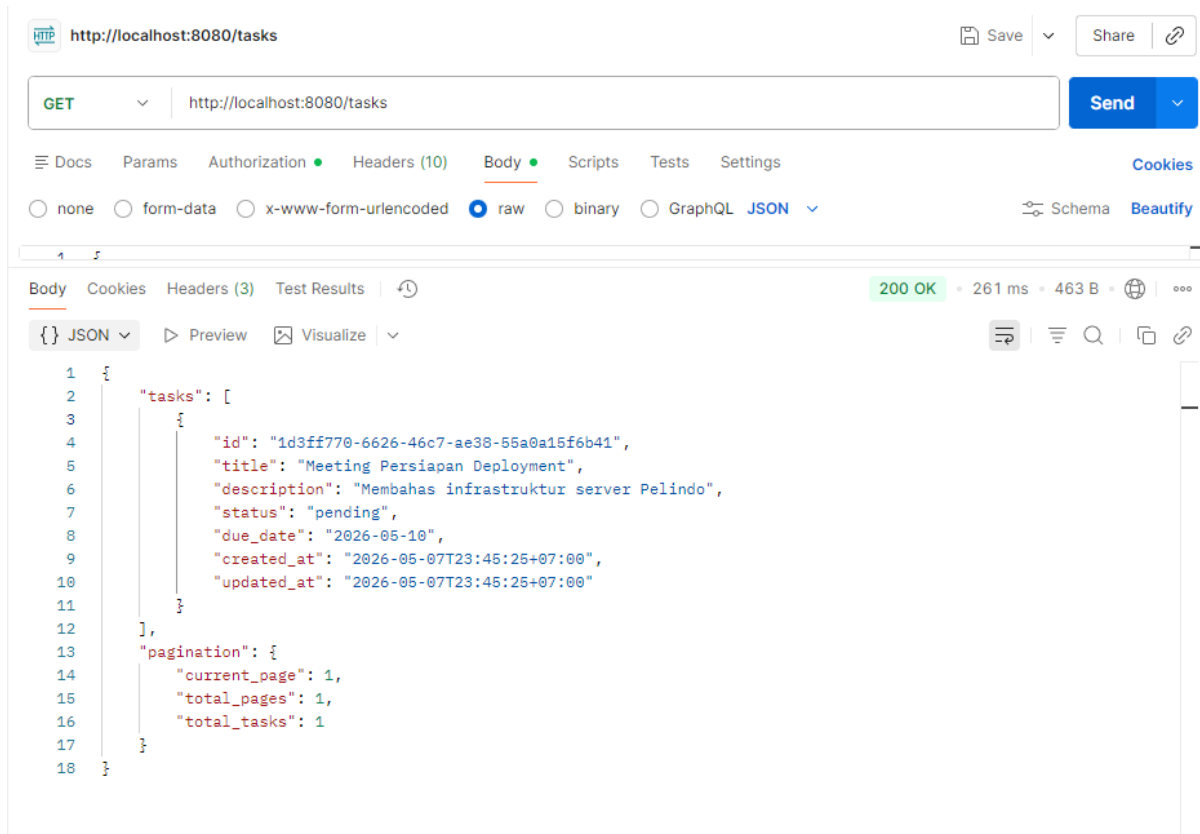
GET /tasks

Header

Authorization: Bearer <TOKEN>

Hasil

Sistem berhasil menampilkan daftar task beserta pagination.



Response

```
{
  "tasks": [
    {
      "title": "Meeting Persiapan Deployment",
      "status": "pending"
    }
  ],
  "pagination": {
    "current_page": 1,
    "total_pages": 1,
  }
```

```
    "total_tasks": 1
  }
}
```

Status

- HTTP Status: 200 OK
- Result: SUCCESS

Catatan

- Endpoint berhasil membaca data task.
 - Redis cache berjalan sesuai mekanisme caching.
 - Filter query dapat digunakan untuk pencarian task.
-

6. Update Task

Endpoint

PUT /tasks/{id}

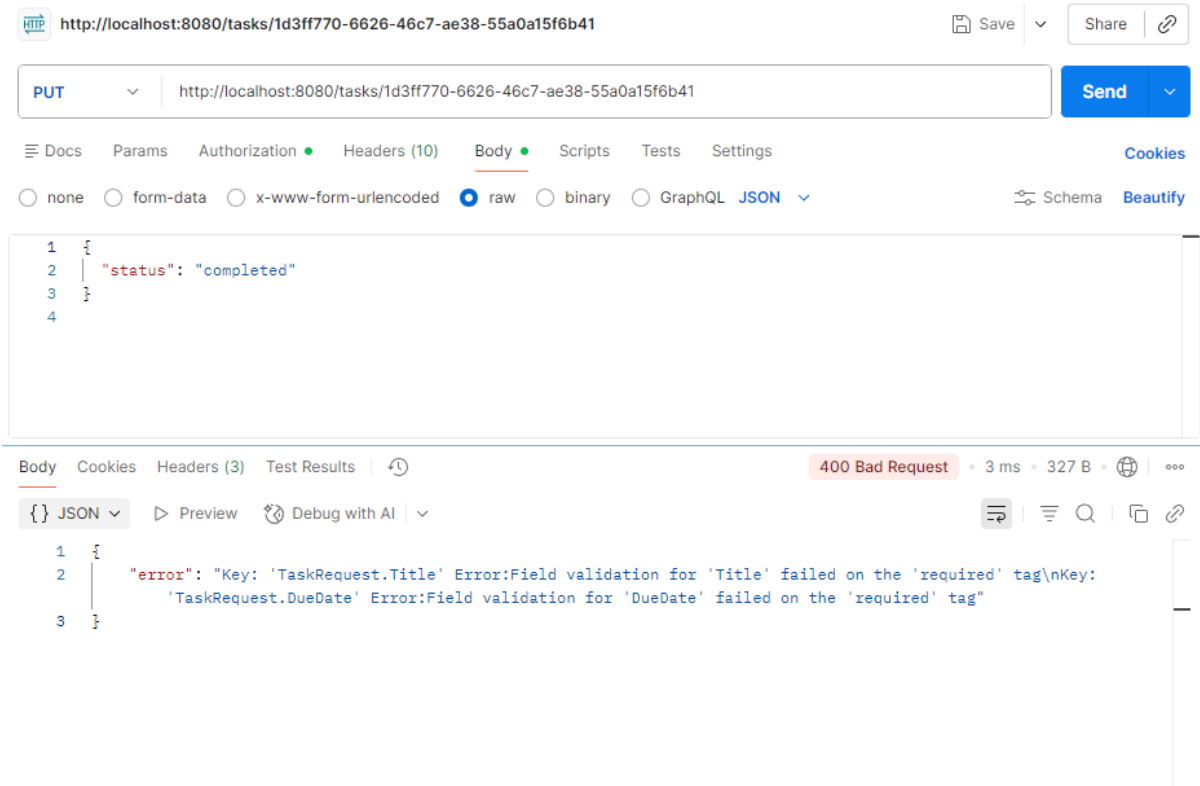
Pengujian Pertama

Request Body

```
{
  "status": "completed"
}
```

Hasil

Request gagal karena field wajib tidak dikirim lengkap.



Response

```
{
  "error": "Field validation failed"
}
```

Status

- HTTP Status: 400 Bad Request
- Result: VALIDATION WORKING

Analysis

Validation pada endpoint update berjalan dengan baik karena sistem mewajibkan field:

- title
- due_date

Pengujian Kedua

Request Body

```
{
  "title": "Meeting Persiapan Deployment",
  "description": "Membahas infrastruktur Deployment",
  "status": "completed",
}
```



```
}
  "due_date": "2026-05-10"
}
```

Hasil

Task berhasil diperbarui.

The screenshot shows a REST client interface with a PUT request to `http://localhost:8080/tasks/1d3ff770-6626-46c7-ae38-55a0a15f6b41`. The request body is a JSON object: `{ "title": "Meeting Persiapan Deployment", "description": "Membahas infrastruktur Deployment", "status": "completed", "due_date": "2026-05-10" }`. The response is a 200 OK status with a JSON body: `{ "message": "Task updated successfully", "task": { "id": "1d3ff770-6626-46c7-ae38-55a0a15f6b41", "title": "Meeting Persiapan Deployment", "description": "Membahas infrastruktur Deployment", "status": "completed", "due_date": "2026-05-10", "created_at": "2026-05-07T23:45:25+07:00", "updated_at": "2026-05-07T23:54:26+07:00" } } }`. The interface includes tabs for Body, Cookies, Headers, Test Results, and a bottom bar with Runner, Start Proxy, Cookies, Vault, and Trash.

Response

```
{
  "message": "Task updated successfully"
}
```

Status

- HTTP Status: 200 OK
- Result: SUCCESS

Catatan

- Redis cache berhasil diinvalidasi setelah update.
- Data task terbaru berhasil tersimpan.

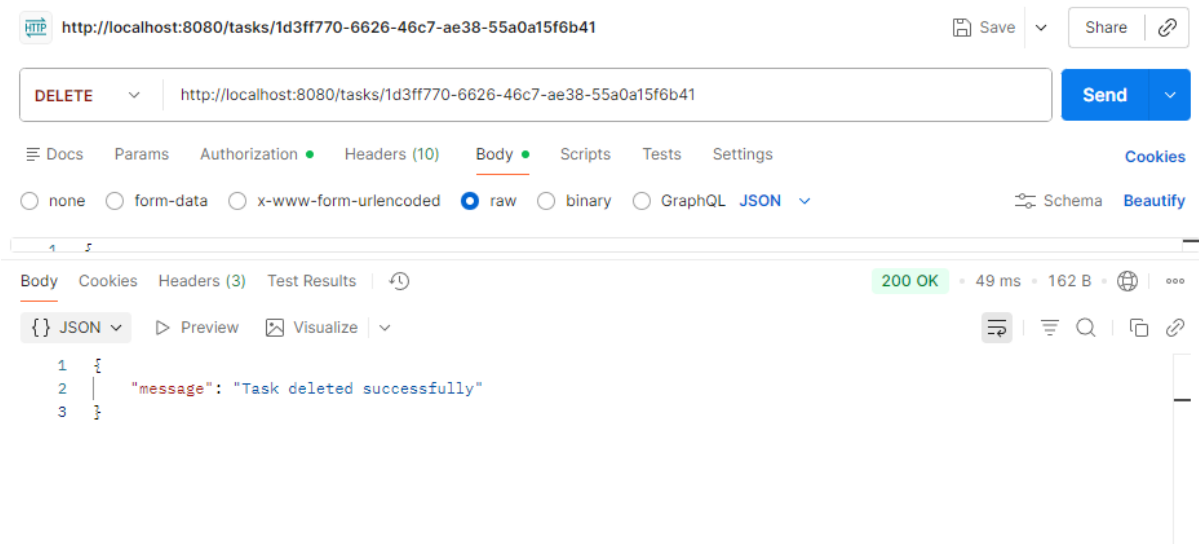
7. Delete Task

Endpoint

DELETE /tasks/{id}

Hasil

Task berhasil dihapus dari sistem.



Response

```
{
  "message": "Task deleted successfully"
}
```

Status

- HTTP Status: 200 OK
- Result: SUCCESS

Catatan

- Redis cache berhasil dihapus setelah proses delete.
- Data task sudah tidak tersedia pada endpoint list.

Kesimpulan

Modul	Status
Health Check	SUCCESS
Register	SUCCESS

Modul	Status
Login JWT	SUCCESS
Create Task	SUCCESS
Get Task	SUCCESS
Update Validation	SUCCESS
Update Task	SUCCESS
Delete Task	SUCCESS
Redis Cache	SUCCESS

Summary

Sistem Task Management API berbasis Golang berhasil berjalan dengan baik meliputi:

- JWT Authentication
- CRUD Task Management
- Request Validation
- Redis Cache Mechanism
- Cache Invalidation
- Pagination dan Filtering

Seluruh endpoint utama berhasil diuji dan berjalan sesuai expected behavior. Validation system pada endpoint update juga berhasil mencegah request tidak valid sehingga keamanan dan konsistensi data tetap terjaga.